

TRABAJO PRÁCTICO INTEGRADOR

Integrantes: Ciro Cattáneo, Franco Gagliardi y Ramiro Quiroga
Capitán: Franco Gagliardi
Profesor: David Roco

Objetivo General

Diseñar e implementar una aplicación orientada a objetos que represente una Baraja de Cartas Españolas, aplicando los conceptos fundamentales de la Programación Orientada a Objetos (POO) mediante el uso de clases, objetos, encapsulamiento y colecciones dinámicas (ArrayList), desarrollando además la capacidad de analizar problemas, modelar soluciones y evaluar la posible aplicación de patrones de diseño según las necesidades del sistema.

Objetivos Específicos

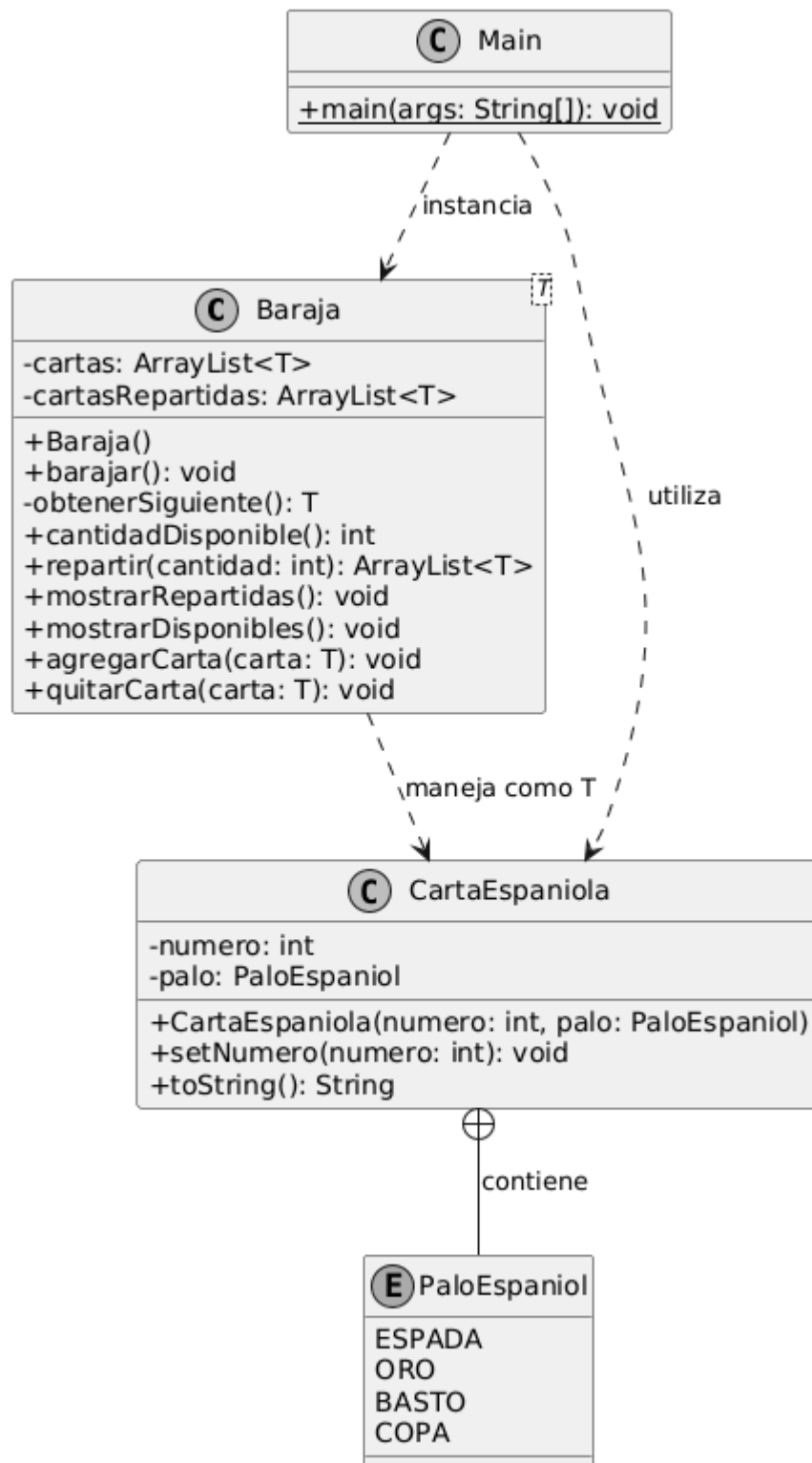
- Identificar las clases necesarias para modelar una baraja española.
- Diseñar atributos y métodos respetando los principios de la Programación Orientada a Objetos.
- Utilizar ArrayList para almacenar y gestionar colecciones de objetos.
- Implementar operaciones de manipulación de la baraja como barajar, repartir y consultar cartas.
- Elaborar un diagrama UML que represente la solución propuesta.
- Analizar las responsabilidades de cada clase dentro del sistema.
- Justificar decisiones de diseño y arquitectura.
- Evaluar críticamente la utilización de patrones de diseño en función del problema planteado.
- Desarrollar buenas prácticas de programación y documentación del código.

1. Análisis del problema

El análisis del problema se centra en simular el comportamiento físico y lógico de una baraja española de 40 cartas bajo el paradigma de la POO, garantizando la consistencia del mazo y evitando el acoplamiento con un juego específico. Para lograrlo, se identificó la necesidad de desacoplar la entidad inmutable CartaEspañola (definida por su número y palo) de la estructura contenedora Baraja<T> (tipo genérico). Esta última se diseñó mediante tipos genéricos para funcionar de manera universal y abstracta, asumiendo únicamente tareas mecánicas globales como almacenar, contar, mezclar y distribuir elementos, sin poseer conocimiento de las reglas de un juego en particular.

Para reflejar la dinámica de una partida real sin sobrecargar a las cartas con variables de estado, el modelo divide el espacio interno del mazo en dos listas dinámicas (cartas disponibles y cartasRepartidas). El flujo de naipes se gestiona mediante un traspaso destructivo y controlado entre listas, asegurando además la atomicidad de las operaciones: el sistema valida el stock antes de repartir y, si la cantidad solicitada supera las existencias, aborta la transacción y revierte el mazo a su estado anterior para evitar que la simulación quede en un estado inconsistente.

2. Diagrama UML



3. Justificación de las decisiones de diseño

El diseño de la solución se estructuró bajo los principios de alta cohesión y bajo acoplamiento mediante el uso de Tipos Genéricos (`Baraja<T>`), abstrayendo el contenedor de la semántica de cualquier naipes o reglamento específico. Esta parametrización otorga al componente una reusabilidad absoluta para futuros desarrollos. Asimismo, para emular la física del movimiento de las cartas sin sobrecargar la entidad `CartaEspañola` con variables de estado, se implementó una estrategia de doble contenedor dinámico (`ArrayList`), donde la transición de naipes entre el mazo de disponibles y el historial de descarte se realiza mediante una extracción destructiva que optimiza el rendimiento y las consultas de cantidad de cartas.

```
public class Baraja<T> { 2 usages  Franco +2
    private ArrayList<T> cartas; 11 usages
    private ArrayList<T> cartasRepartidas; 6 usages

    public Baraja() { 1 usage  Franco
        this.cartas = new ArrayList<>();
        this.cartasRepartidas = new ArrayList<>(); //lista donde estan las cartas que ya se repartieron
    }

    //Mezcla las cartas
    public void barajar() { 2 usages  Franco +1
        // Se devuelven al mazo todas las cartas usadas previamente
        cartas.addAll(cartasRepartidas);
        // Se vacia la lista de repartidas
        cartasRepartidas.clear();
        // Se mezclan todas las cartas aleatoriamente
        Collections.shuffle(cartas);
    }

    //Devuelve la siguiente carta disponible
    private T obtenerSiguiente() { 1 usage  Ramiro Quiroga +1
        if (cartas.isEmpty()) {
            return null; // En caso de no haber más cartas en el mazo, retorna null
        }
        return cartas.remove(index: 0); // Saca la carta de arriba (posición 0) y la devuelve
    }
}
```

código de la clase baraja

Por otra parte, se priorizó el ocultamiento de información y la robustez del sistema al definir el método `obtenerSiguiente()` con visibilidad privada, centralizando la extracción únicamente a través de la función pública `repartir(int cantidad)`. Esto garantiza la atomicidad de las operaciones bajo una política de "todo o nada": si el stock remanente es insuficiente para completar una mano, el sistema revierte los

cambios pendientes y aborta la transacción de forma segura devolviendo null. Finalmente, la integridad de los datos quedó blindada en la entidad mínima mediante un método setter defensivo para el rango numérico y el uso de un tipo enumerado (enum) para los palos, eliminando errores de ejecución en tiempo de compilación.

```
//Devuelve una lista con cierta cantidad de cartas
public ArrayList<T> repartir(int cantidad){ 2 usages 2 Franco
    ArrayList<T> cartasARepartir = new ArrayList<>(); //Creo una lista con las cartas que voy a repartir
    for (int i = 1; i <= cantidad; i++) { //Itero en base a la cantidad de cartas que quiero repartir
        T siguiente = obtenerSiguiente();
        if (siguiente != null){ //Si todavia no se acaban las cartas sigo repartiendo
            cartasARepartir.add(siguiente);
        } else { //Si se acaban devuelvo null para indicar que no se pudieron repartir
            System.out.println("Ya no quedan cartas suficientes");
            cartas.addAll(cartasARepartir);
            return null; //Esto hace que si solo quedan por ejemplo 2 cartas para repartir pero yo queria repartir 4 me devuelva null
        }
    }
    cartasRepartidas.addAll(cartasARepartir); //Pongo los elementos de cartasARepartir en cartasRepartidas
    return cartasARepartir;
}
```

método repartir

4. Análisis sobre posibles patrones de diseño aplicables.

Dada la arquitectura modular de la solución, el sistema es idóneo para incorporar patrones de diseño de *GoF (Gang of Four)* que permitan escalar sus capacidades sin corromper la estructura existente. El análisis enfatiza la aplicabilidad del patrón creacional *Factory Method*, el cual permitiría delegar la construcción y el filtrado reglamentario de las cartas (como la omisión de los números 8 y 9) a una fábrica especializada (*BarajaEspañolaFactory*), liberando al módulo *Main* de los bucles de inicialización explícitos.

```

public class CartaEspaniola { 7 usages  Franco +1
    private int numero; 2 usages
    private PaloEspaniol palo; 2 usages

    public CartaEspaniola(int numero, PaloEspaniol palo) { 1 usage  Franco
        setNumero(numero);
        this.palo = palo;
    }

    public enum PaloEspaniol { 4 usages  Franco
        ESPADA, no usages
        ORO, no usages
        BASTO, no usages
        COPA; no usages
    }

    public void setNumero(int numero) { 1 usage  Franco
        if (numero > 0 && numero <= 12){
            this.numero = numero;
        }
    }

    @Override  Franco-Cattaneo
    public String toString() { return numero + " de " + palo; }
}

```

clase cartaEspaniola

Asimismo, el comportamiento de la mezcla podría flexibilizarse mediante el patrón Strategy, aislando el algoritmo de barajado en clases independientes para admitir diferentes reglamentos en tiempo de ejecución. Por último, la implementación del patrón Iterator (Iterable<T>) blindaría los encapsulamientos privados, permitiendo a componentes externos recorrer de forma secuencial las cartas disponibles o repartidas sin necesidad de conocer o acceder directamente a las estructuras ArrayList internas.

```

// 1. Instanciar la baraja genérica especificando que es de CartaEspaniola
Baraja<CartaEspaniola> mazo = new Baraja<>();

// 2. Inicializar el mazo con las 40 cartas reglamentarias (excluyendo 8 y 9)
for (CartaEspaniola.PaloEspaniol palo : CartaEspaniola.PaloEspaniol.values()) {
    for (int num = 1; num <= 12; num++) {
        if (num != 8 && num != 9) {
            mazo.agregarCarta(new CartaEspaniola(num, palo));
        }
    }
}
}

```

código para inicializar el mazo

Es importante señalar que, aunque el desarrollo actual no fue realizado inicialmente bajo el uso explícito de estas estructuras debido al desconocimiento previo sobre el concepto de patrones de diseño, el análisis de estos, denota su enorme utilidad práctica. Comprender estos patrones no solo proporciona un vocabulario técnico

unificado para el trabajo en equipo, sino que ofrece soluciones probadas a problemas recurrentes en el desarrollo de software. Reconocer su aplicabilidad en este punto del proyecto abre una valiosa perspectiva de mejora continua, demostrando cómo el código base puede evolucionar hacia un sistema mucho más flexible, mantenible y preparado para requerimientos más complejos en el futuro.